

Qué es, para qué sirve y qué valor usar

Los valores que recomiendan los cuadernos, cada término explicado en una línea, las importaciones y lo que los cuadernos marcan como importante.



Los números que los cuadernos dan como punto de partida

Qué	Valor recomendado	Por qué / nota
Tasa de aprendizaje η	0.001 a 0.1; empezar en 0.01 o 0.1	≥ 0.5 oscila y no converge; ≤ 0.0001 aprende muy lento
Adam (learning_rate)	0.001 (default de Keras)	El optimizador más usado en los cuadernos
Épocas	Empezar con 100 o 500	Si el error sigue bajando, aumentar
batch_size	16, 32, 64 o 128; empezar con 32 o 64	Potencias de 2; muy grande generaliza peor, muy pequeño es inestable
Neuronas ocultas	Potencias de 2: 32, 64, 128, 256	Ir reduciendo: 128 $\rightarrow$ 64 $\rightarrow$ 32 (embudo)
Umbral de clase	0.5	clase = 1 si probabilidad > 0.5
Dropout	0.2 (C3, C5) · 0.5 (C4, C8)	Fracción de neuronas que se apagan al entrenar
Regularización L2	l2(0.01)	Penaliza pesos grandes

División de datos, early stopping y sklearn

Tamaño del dataset	Train	Val.	Test
Grande	70%	15%	15%
Mediano	70%	20%	10%
Pequeño	80%	10%	10%

En el código	Valor
test_size	0.2 (80% / 20%)
random_state	42 (reproducibilidad)
validation_split	0.2

EarlyStopping	Valor recomendado
monitor	'val_loss'
patience	3 a 10 (5 en el ejemplo)
min_delta	0.001
restore_best_weights	True (siempre en producción)
mode	'min' pérdida · 'max' accuracy

Perceptron sklearn	Default
max_iter	1000
eta0	1.0
tol	1e-3 (0.001)

Rangos y tamaños que no cambian

Qué	Valor	Nota
Píxel uint8	0 a 255	0 = negro, 255 = blanco
Normalizar imagen	dividir entre 255.0	Deja los valores en [0, 1]
Pesos escala de grises	0.299 R · 0.587 G · 0.114 B	El verde pesa más (ojo más sensible)
Kernel	3×3 el más común; también 5×5, 7×7	Si suma 1, conserva el brillo
MaxPooling	2×2	Divide alto y ancho entre 2
CIFAR-10	32×32×3, 10 clases, 50 000 train / 10 000 test	Imágenes a color pequeñas
VGG16 / ResNet50 · InceptionV3	224×224 · 299×299	Tamaño de entrada de modelos preentrenados
Recall en enfermedades graves	90% o más	El mínimo lo deciden médicos y reguladores

Las piezas básicas de una red

Perceptrón

Neurona más simple (Rosenblatt, 1958).
Clasifica en 2 clases con una recta

Entradas (x)

Características del dato que entran a la neurona (ej. nota, edad)

Peso (w)

Importancia de cada entrada; es lo que la red ajusta al aprender

Sesgo (b, bias)

Umbral que desplaza la frontera; deja decidir aunque x sea 0

Suma ponderada (z)

$z = w \cdot x + b$: combina entradas y pesos antes de activar

Función de activación

Transforma z en la salida; las no lineales dan poder a la red

Linealmente separable

Las clases se pueden dividir con una recta o hiperplano

XOR

Ejemplo clásico que NO es linealmente separable: un perceptrón no puede

MLP

Perceptrón multicapa: varias capas que resuelven problemas no lineales

Capa densa (Dense)

Cada neurona conectada con todas las de la capa anterior

Capa oculta

Capa entre la entrada y la salida; extrae patrones

Parámetros

Pesos y sesgos que la red aprende sola

Cómo aprende la red

Época	Una pasada completa por todos los datos de entrenamiento
Batch (lote)	Grupo de datos que se procesa antes de actualizar los pesos
Mini-batch	Punto medio entre todo el dataset (Batch GD) y una muestra (SGD)
Tasa de aprendizaje (η)	Tamaño del paso con que se corrigen los pesos
Convergencia	Cuando el error deja de bajar y se estabiliza
Hiperparámetro	Lo eliges tú antes de entrenar: η , épocas, batch, capas

Función de pérdida	Mide qué tan mal predice; la red la minimiza al entrenar
Métrica	Solo evalúa el desempeño (accuracy, recall); no entrena
Gradiente	Derivada de la pérdida: dice hacia dónde sube el error
Descenso del gradiente	Mover los pesos en contra del gradiente para bajar el error
Backpropagation	Propaga el error hacia atrás para calcular los gradientes
Optimizador	Algoritmo que actualiza los pesos: SGD, Adam, RMSprop

Lo que sale mal al entrenar y cómo se arregla

Overfitting

Memoriza el entrenamiento y falla con datos nuevos (loss ↓, val_loss ↑)

Underfitting

No aprende lo suficiente: pocas épocas o modelo muy simple

Generalización

Que funcione bien con datos que nunca vio

Vanishing gradient

Gradientes casi cero que frenan el aprendizaje (sigmoid, tanh)

Data leakage

Fuga de datos: información de test se cuela al entrenar

Desbalance de clases

Una clase es mucho más frecuente; accuracy engaña

Early stopping

Para el entrenamiento cuando la validación deja de mejorar

patience

Épocas que espera sin mejora antes de parar

restore_best_weights

Al parar, vuelve a los pesos de la mejor época

Dropout

Apaga neuronas al azar al entrenar para evitar overfitting

Regularización L2

Castiga los pesos grandes para simplificar el modelo

Data augmentation

Crear datos nuevos girando o volteando imágenes

Qué es ReLU y las demás, y dónde va cada una

ReLU

$$\max(0, x)$$

Qué hace: Si x es negativo da 0; si es positivo lo deja igual. Rápida, reduce el vanishing gradient

Dónde: Capas ocultas. La opción por defecto

Sigmoid

$$1/(1+e^{-x})$$

Qué hace: Aplasta cualquier número entre 0 y 1: se lee como probabilidad

Dónde: Salida de clasificación binaria

Tanh

rango $(-1, 1)$

Qué hace: Como sigmoid pero centrada en 0, gradientes más fuertes

Dónde: Ocultas con datos centrados; RNN

Linear

$$f(x) = x$$

Qué hace: No transforma nada. Es la que Keras usa si no pones activation

Dónde: Solo en la salida de regresión

Softmax

$$e^{z_i} / \sum e^{z_j}$$

Qué hace: Convierte varios números en probabilidades que suman 1

Dónde: Salida multiclase (una clase por dato)

Escalón

1 si $z > 0$, si no 0

Qué hace: Decide 0 o 1 de golpe. Es la del perceptrón original

Dónde: Solo el perceptrón (C1)

Cuál usar según el problema

MSE	Error cuadrático medio. Castiga mucho los errores grandes. Regresión	Binary crossentropy	Para 2 clases con salida sigmoid; castiga la confianza equivocada
MAE	Error absoluto medio: el error promedio directo. Menos sensible a outliers	Categorical crossentropy	Multiclase con etiquetas one-hot ([0,0,1,0])
RMSE	Raíz del MSE: el error en las mismas unidades que lo que predices	Sparse categorical	Multiclase con etiquetas enteras (0, 1, 2...)
Outlier	Dato atípico, muy alejado del resto	from_logits=True	La salida no tiene softmax; la pérdida lo aplica por dentro

Problema	Salida	Pérdida
Regresión	1 neurona linear	mse / mae
Binaria	1 neurona sigmoid	binary_crossentropy
Multiclase	N neuronas softmax	categorical o sparse_categorical

MATRIZ DE CONFUSIÓN

Verdadero o falso dice si acertó; positivo o negativo, qué predijo

VP • Verdadero positivo

Dijo SÍ y era SÍ

Dice enfermo y el paciente sí está enfermo. Acierto.

FN • Falso negativo

Dijo NO y era SÍ

Dice sano pero está enfermo: se va sin tratamiento. El más grave en medicina.

FP • Falso positivo

Dijo SÍ y era NO

Dice enfermo pero está sano: falsa alarma, exámenes de más.

VN • Verdadero negativo

Dijo NO y era NO

Dice sano y sí está sano. Acierto.

$Accuracy = (VP+VN) / total$
 $Precision = VP / (VP+FP)$
 $Recall = VP / (VP+FN)$

Accuracy: de todo, cuánto acertó.

Precision: cuando dice SÍ, qué tan seguido acierta. Baja si hay muchas falsas alarmas (FP).

Recall (sensibilidad): de los SÍ reales, cuántos encontró. Baja si se le escapan (FN).

- 0 En medicina importa más el **recall**: es mejor una falsa
- 3 alarma que un enfermo sin detectar.
- 0

Truco para leer el nombre: la segunda palabra (positivo/negativo) es lo que PREDIJO el modelo; la primera (verdadero/falso) dice si acertó. Falso negativo = predijo negativo y se equivocó. Con clases desbalanceadas un modelo que dice 'sano' a todos tiene accuracy alto pero recall 0. El cuaderno 1 menciona también la curva ROC y el AUC (área bajo la curva) como medida adicional.

Limpiar, dividir y escalar antes de entrenar

Train / Validation / Test	Train aprende · Validation ajusta hiperparámetros · Test evalúa al final	Normalizar / escalar	Poner las variables en rangos parecidos para que aprenda mejor
train_test_split	Divide los datos en entrenamiento y prueba	MinMaxScaler	Lleva cada columna a [0, 1]: $(x - \min) / (\max - \min)$
stratify=y	Mantiene la misma proporción de clases en train y test	StandardScaler	Media 0 y desviación 1: $(x - \mu) / \sigma$
random_state	Semilla: da siempre la misma división (reproducible)	fit_transform / transform	fit aprende min, max o media de TRAIN; transform solo aplica
K-Fold	Divide en K partes y entrena K veces; para datasets pequeños	OneHotEncoder	Convierte una categoría en columnas de 0 y 1
dropna	Borra las filas con valores nulos	Imputar	Rellenar nulos: media o mediana (números), moda (categorías)

Qué se importa y para qué sirve

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import joblib
from sklearn.linear_model import Perceptron
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler,
StandardScaler, OneHotEncoder
from sklearn.metrics import accuracy_score, recall_score
from sklearn.datasets import make_circles
```

```
import tensorflow as tf
from tensorflow.keras import layers, models, Sequential, Input
from tensorflow.keras.layers import Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping
```

numpy / pandas Arreglos y matrices / tablas (DataFrame)

matplotlib Gráficas e imágenes

joblib Guardar y cargar modelos o escaladores

Perceptron El perceptrón ya hecho

make_circles Crea los datos de círculos concéntricos

tensorflow / keras Crear, entrenar y guardar redes

EarlyStopping Callback para parar a tiempo

Librerías de los cuadernos 6 y 7

```
import cv2 as cv
from PIL import Image
from scipy.signal import convolve2d
from scipy.ndimage import zoom, sobel,
uniform_filter, median_filter
from sklearn.cluster import KMeans
from skimage import measure
from tensorflow.keras import datasets
from tensorflow.keras.preprocessing import image
from tensorflow.keras.models import load_model
```

cv2 (OpenCV)	Leer (imread), guardar (imwrite), convertir color, filtrar
PIL	Abrir y redimensionar imágenes
convolve2d	Aplicar un kernel a una imagen
zoom / sobel	Reducir tamaño / detectar bordes
KMeans	Segmentar agrupando colores
datasets	Traer CIFAR-10 listo
image	img_to_array, load_img
load_model	Cargar un modelo guardado

Métodos y objetos que más aparecen

Sequential	Modelo de capas en fila: la salida de una entra a la siguiente	Tensor	Arreglo multidimensional de TensorFlow, puede ir a GPU
Input(shape=...)	Declara la forma de los datos de entrada	shape / dtype	Tamaño en cada eje / tipo de dato (float32)
compile()	Elige optimizador, pérdida y métricas antes de entrenar	tf.constant / tf.Variable	Valor fijo / valor que cambia (los pesos)
fit()	Entrena el modelo; devuelve history con la pérdida por época	axis=0 / axis=1	Opera por columna / opera por fila
evaluate() / predict()	Mide pérdida y métrica / predice datos nuevos	GradientTape	Graba operaciones y calcula derivadas automáticamente
summary()	Muestra capas, formas y número de parámetros	@tf.function	Convierte la función en grafo más rápido
save() / .keras	Guarda el modelo; .keras es el formato recomendado	verbose	0 nada, 1 barra de progreso, 2 una línea por época

Cuaderno 6 en palabras simples

Píxel	Cada número de la matriz de la imagen	Kernel o filtro	Matriz pequeña que recorre la imagen para transformarla
Canal	Capa de color: R, G o B. Gris tiene 1, color tiene 3	Convolución	Multiplicar el kernel con la vecindad y sumar → nuevo píxel
BGR	Orden en que OpenCV lee los colores (al revés de RGB)	Desenfoque (media, gauss)	Suaviza la imagen promediando vecinos
Escala de grises	Imagen 2D de un solo valor por píxel	Mediana	Quita ruido y conserva los bordes
Imagen binaria	Solo dos valores: blanco y negro	Sobel	Detecta bordes: cambios bruscos de intensidad
flatten / ravel	Aplanar a 1D (copia / vista); lo necesita la capa densa	Umbral · Otsu · K-Means	Separar regiones: corte fijo, corte automático, grupos de color

Cuaderno 7 en palabras simples

CNN	Red que aprende sus propios kernels para ver patrones en imágenes	Logits	Salida sin softmax: números crudos, no probabilidades
Conv2D	Capa convolucional: aplica varios filtros y crea mapas de características	argmax	Índice del valor más alto = la clase predicha
Filtros	Cuántos kernels aprende la capa (32, 64...) = canales de salida	expand_dims	Añade la dimensión de lote: (32,32,3) → (1,32,32,3)
Padding / stride	Borde añadido / salto del kernel. En Keras: sin borde y paso 1	CIFAR-10	Dataset de 60 000 imágenes 32×32 a color en 10 clases
MaxPooling	Toma el máximo de cada bloque 2×2: reduce tamaño, sin parámetros	.h5 · .tfLite · JSON	Formato antiguo · para móviles · solo arquitectura
Flatten	Aplana el resultado 3D a un vector para las capas densas	Transfer learning (C8)	Reusar un modelo ya entrenado y entrenar solo la parte nueva

Orden típico de una CNN: Conv2D + ReLU → MaxPooling (repetir) → Flatten → Dense → salida. Al avanzar, alto y ancho bajan y los filtros suben. Una CNN detecta características locales (bordes, esquinas, texturas) y aprende de forma jerárquica.

LO QUE LOS CUADERNOS MARCAN COMO IMPORTANTE

Doce ideas clave para llevar al examen

01 Usa la misma versión de TensorFlow para entrenar y para desplegar (C2)

03 Elegir mal la pérdida impide que el modelo aprenda, aunque la red esté bien (C4)

05 Bajo error en train no es un buen modelo: importa el desempeño en datos no vistos (C4)

07 El escalador se ajusta solo con train; en test solo transform (C1.1, C5)

09 Un perceptrón solo separa clases con una recta; los círculos necesitan capas ocultas (C1, C4)

11 La capa densa solo recibe vectores planos: por eso Flatten (C6, C7)

02 No uses linear en todas las capas ocultas: equivale a una sola capa (C4)

04 Train entrena, Validation ajusta, Test evalúa. Nunca ajustes con test (C4)

06 EarlyStopping no mejora el modelo: conserva su mejor punto. Monitorea val_loss (C4)

08 En series de tiempo no mezcles datos futuros con pasados (C4)

10 OpenCV lee BGR; imshow con float espera 0–1 y con uint8 0–255 (C6)

12 Para predecir, prepara la imagen igual que en el entrenamiento: tamaño, /255 y lote (C7)